

CBCS SCHEME

First/ Second Semester B.E Degree Examination, 2025-26

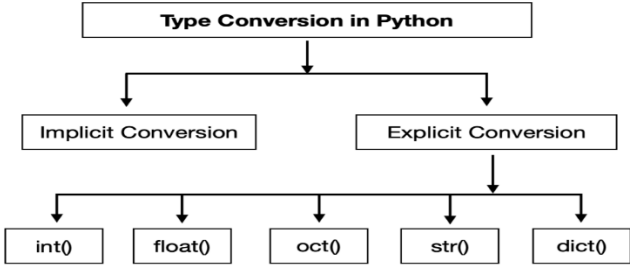
PYTHON PROGRAMMING (1BPLC105B/205B)

TIME: 03 Hours

Max. Marks:100

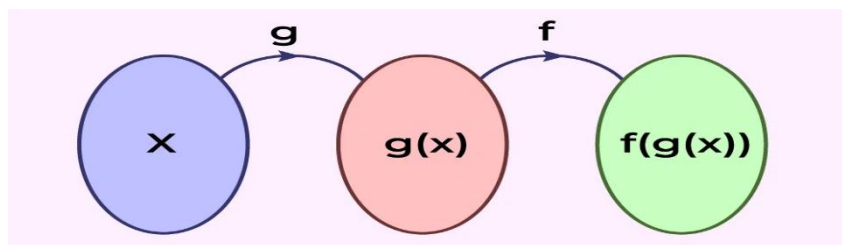
Notes:

1. Answer any FIVE full questions, choosing at least ONE question from each MODULE
2. M: Marks, L: Bloom's level, C: Course outcomes.
- 3.

	Module - 1	M	L	C
Q.1	a Explain the concept of type conversion in Python. Differentiate between implicit and explicit conversion with examples. Type Conversion • Type conversion is the process of converting one data type into another data type. • Python supports automatic and manual type conversion. 1) Implicit Type Conversion • It is done automatically by Python. • No built-in function is used. • It happens when different data types are used in an expression. • Python converts lower data type into higher data type to avoid data loss. • Example: When int and float are added, result becomes float. 2) Explicit Type Conversion • It is done manually by the programmer. • Built-in functions are used. • Common functions: int(), float(), str(). • It gives more control over conversion. • Example: Converting string to integer using int() function.  <pre> graph TD A[Type Conversion in Python] --> B[Implicit Conversion] A --> C[Explicit Conversion] C --> D[int()] C --> E[float()] C --> F[oct()] C --> G[str()] C --> H[dict()] </pre>	08	L2	CO1

		<table><tr><th>Implicit Conversion</th><th>Explicit Conversion</th></tr><tr><td>Automatic conversion</td><td>Manual conversion</td></tr><tr><td>No function used</td><td>Uses conversion functions</td></tr><tr><td>Done by compiler automatically</td><td>Done by programmer</td></tr><tr><td>No data loss (generally)</td><td>May cause data loss</td></tr><tr><td>Example: int → float</td><td>Example: (int) 10.5</td></tr></table>	Implicit Conversion	Explicit Conversion	Automatic conversion	Manual conversion	No function used	Uses conversion functions	Done by compiler automatically	Done by programmer	No data loss (generally)	May cause data loss	Example: int → float	Example: (int) 10.5			
Implicit Conversion	Explicit Conversion																
Automatic conversion	Manual conversion																
No function used	Uses conversion functions																
Done by compiler automatically	Done by programmer																
No data loss (generally)	May cause data loss																
Example: int → float	Example: (int) 10.5																
b	Develop a Python program with a while loop to display the Fibonacci sequence up to n terms entered by the user. <pre>n = int(input("Enter number of terms: ")) a = 0 b = 1 count = 0 while count < n: print(a) c = a + b a = b b = c count = count + 1</pre> 1. Read the number of terms from the user. 2. Initialize first two numbers as 0 and 1. 3. Initialize counter variable count = 0. 4. Use while loop to repeat until count < n. 5. Print the current Fibonacci number. 6. Calculate next number by adding previous two numbers. 7. Update values of a and b. 8. Increment the counter in each iteration.	08	L3	CO1													
c	Differentiate between a syntax error and a runtime error with examples. <table><tr><th>Syntax Error</th><th>Runtime Error</th></tr><tr><td>Occurs when Python rules are not followed</td><td>Occurs during execution of the program</td></tr><tr><td>Program does not execute</td><td>Program starts but stops in between</td></tr><tr><td>Detected before execution</td><td>Detected during execution</td></tr><tr><td>Example: Missing colon or bracket</td><td>Example: Division by zero</td></tr></table>	Syntax Error	Runtime Error	Occurs when Python rules are not followed	Occurs during execution of the program	Program does not execute	Program starts but stops in between	Detected before execution	Detected during execution	Example: Missing colon or bracket	Example: Division by zero	04	L2	CO1			
Syntax Error	Runtime Error																
Occurs when Python rules are not followed	Occurs during execution of the program																
Program does not execute	Program starts but stops in between																
Detected before execution	Detected during execution																
Example: Missing colon or bracket	Example: Division by zero																
OR																	
Q.2	a	Describe the Collatz $3n + 1$ sequence and explain how iteration and conditional statements are used in its implementation. Program: <pre>n = int(input("Enter a positive number: ")) while n != 1: print(n) if n % 2 == 0: n = n // 2 else:</pre>	08	L2	CO1												

	<pre>n = 3 * n + 1</pre> <pre>print(1)</pre> Explanation: 1. Read a positive integer from the user. 2. Use a while loop that runs until n becomes 1. 3. Print the current value of n. 4. Use if-else statement for decision making: - o If n is even ($n \% 2 == 0$), divide it by 2. - o If n is odd, multiply by 3 and add 1. 5. Repeat the process until n becomes 1. 6. Finally, print 1 as the last value in the sequence.			
	Output: <pre>Enter a positive number: 6 6 3 10 5 16 8 4 2 1</pre>			
	b Develop a program that prints all numbers from 1 to 100 that are divisible by 3 or 5 but not both. Use continue or break statements wherever suitable. Program: <pre>for i in range(1, 101): if (i % 3 == 0 or i % 5 == 0) and not (i % 3 == 0 and i % 5 == 0): print(i)</pre> Explanation: 1. Use a for loop to iterate numbers from 1 to 100. 2. Check if the number is divisible by 3 using $i \% 3 == 0$. 3. Check if the number is divisible by 5 using $i \% 5 == 0$. 4. Use logical operators: - o or $\rightarrow$ divisible by 3 or 5. - o and not $\rightarrow$ ensure it is not divisible by both. 5. Print the numbers that satisfy the condition.	08	L3	CO1
	c What is meant by function composition? Illustrate with an example. • Function composition means combining two or more functions. • The output of one function becomes the input of another function. • If $f(x)$ and $g(x)$ are two functions, then $f(g(x))$ is called function composition. • It helps in writing modular and reusable programs.	04	L2	CO1



Example:

If

$$f(x) = x + 2$$

$$g(x) = x \times 3$$

Then

$$f(g(x)) = f(3x) = 3x + 2$$

Module – 2

Q.3

a Explain the string operations in Python for slicing, concatenation, repetition, and comparison with suitable examples.

8

L2

CO2

1) Slicing

- Slicing is used to extract a part of a string.
- Syntax: string[start:end]
- It returns characters from start index to end-1 index.
- Example:
If s = "Python"
s[0:4] gives "Pyth"

2) Concatenation

- Concatenation means joining two or more strings.
- + operator is used.
- Example:
"Hello" + "World" gives "HelloWorld"

3) Repetition

- Repetition means repeating a string multiple times.
- * operator is used.
- Example:
"Hi" * 3 gives "HiHiHi"

4) Comparison

- Strings can be compared using comparison operators.
- Operators: ==, !=, <, >
- Comparison is based on ASCII values.
- Example:
"apple" == "apple" → True
"abc" < "bcd" → True

b Define a list. How is it different from an array? Develop a Python 6 statement to access the third element of a list: nums = [3, 6, 9, 12].

6

L3

CO2

- A list is a collection of elements enclosed in square brackets [].
- It can store different types of data.
- Lists are mutable.

		Example: nums = [3, 6, 9, 12] Difference Between List and Array - List can store different data types. - Array stores same data type elements. - List is built-in in Python. - Array requires importing array module. - List size can change dynamically. Access Third Element - Index starts from 0. - Third element index = 2 Statement: nums[2]			
	c	Develop a program to count the number of words in a given line of text. <pre>text = input("Enter a line: ") words = text.split() count = len(words) print("Number of words:", count)</pre> - Read a line from user. - split() divides sentence into words. - len() counts number of words. - Print the result.	6	L3	CO2
OR					
Q.4	a	Explain mutability in lists. Illustrate the difference between modifying a list and creating a clone of it using examples. Mutability - Mutability means data can be changed after creation. - Lists are mutable in Python. - We can add, remove, or modify elements. Example: nums = [1, 2, 3] nums[0] = 10 Now list becomes [10, 2, 3] Modifying a List - Changes original list. - Example: a = [1, 2, 3] b = a b[0] = 100 Both a and b become [100, 2, 3] Creating a Clone of List - Creates a separate copy.	8	L2	CO2

	- Original list does not change. - Example: a = [1, 2, 3] b = a.copy() b[0] = 100 Now: - a = [1, 2, 3] - b = [100, 2, 3]			
	b Develop a Python program to check if a string is a palindrome using slicing. <pre>text = input("Enter a string: ") if text == text[::-1]: print("Palindrome") else: print("Not Palindrome")</pre> Explanation [::-1] reverses the string. - Compare original and reversed string. - If both are same → Palindrome.	6	L3	CO2
	c Develop a program that takes a list of numbers and returns a new list containing only the even numbers. <pre>nums = [int(x) for x in input("Enter numbers: ").split()] even_list = [] for n in nums: if n % 2 == 0: even_list.append(n) print("Even numbers:", even_list)</pre> Explanation - Read list of numbers. - Check each number using modulus operator. - If divisible by 2, add to new list. - Print even numbers list.	6	L3	CO2
Module – 3				
Q5	a Develop a Python program that counts the frequency of words in a paragraph using a dictionary and displays the top three most frequent words. <pre>text = input("Enter a paragraph: ") words = text.split() freq = {}</pre>	8	L3	CO2

	<pre> for word in words: word = word.lower() if word in freq: freq[word] += 1 else: freq[word] = 1 sorted_words = sorted(freq.items(), key=lambda x: x[1], reverse=True) print("Top three frequent words:") for i in range(3): print(sorted_words[i]) </pre> Explanation • Read paragraph from user. • Split paragraph into words. • Use dictionary to store word and its frequency. • Convert words to lowercase for accuracy. • Sort dictionary based on frequency in descending order. • Display top three words.			
b What is masking in NumPy? Develop a program to illustrate masking to filter array elements.	<pre> import numpy as np arr = np.array([10, 25, 30, 45, 50]) mask = arr > 30 filtered = arr[mask] print("Original array:", arr) print("Mask:", mask) print("Filtered elements:", filtered) </pre> Explanation • Create NumPy array. • Apply condition (arr > 30). • Boolean mask is created. • Only elements greater than 30 are selected.	6	L3	CO3
c Explain the use of the 'with' statement in file handling with a program.	• The with statement is used to open files. • It automatically closes the file after use. • It avoids manual closing using close() method. • It improves safety and readability. Program <pre> with open("sample.txt", "w") as f: </pre>	6	L2	CO4

	<pre>f.write("Hello Python")</pre> with open("sample.txt", "r") as f: content = f.read() print(content) Advantages • File closes automatically. • Prevents file corruption. • Cleaner and safer code.			
OR				
	a Explain the key features and operations of Python dictionaries. How are they different from lists? Develop suitable program to illustrate insertion, deletion, and lookup. • Dictionary stores data in key-value pairs. • Written inside { }. • Keys must be unique. • Values can be any data type. • Dictionaries are mutable. Example: student = {"name": "Ram", "age": 20} Difference Between Dictionary and List • Dictionary stores key-value pairs. • List stores ordered elements. • Dictionary accessed using keys. • List accessed using index. • Dictionary uses { }. • List uses []. Program for Insertion, Deletion and Lookup <pre>student = {} # Insertion student["name"] = "Ram" student["age"] = 20 # Lookup print(student["name"]) # Deletion del student["age"] print(student)</pre>	8	L3	CO2
	b Develop a NumPy program to: Create a 3×3 matrix of random integers. Display its shape, transpose, and mean of all elements.	6	L3	CO3

	<pre>import numpy as np # Create 3x3 random integer matrix matrix = np.random.randint(1, 50, (3,3)) print("Matrix:\n", matrix) # Display shape of matrix print("Shape:", matrix.shape) # Display transpose of matrix print("Transpose:\n", matrix.T) # Display mean of all elements print("Mean:", matrix.mean())</pre> Explanation: 1. Import NumPy library using import numpy as np. 2. Create a 3×3 random integer matrix using np.random.randint(1, 50, (3,3)). 3. .shape returns the dimensions (rows, columns) of the matrix. 4. .T gives the transpose of the matrix (rows become columns). 5. .mean() calculates the average of all elements in the matrix. 6. The results are printed using print() function.			
--	---	--	--	--

c Explain how binary files differ from text files in terms of content and operations. Illustrate with suitable program segments. Text Files • Stores data in readable form. • Stores characters. • Example: .txt files. • Uses modes like "r", "w". Example: with open("file.txt", "w") as f: f.write("Hello") Binary Files • Stores data in binary format (0 and 1). • Not human readable. • Used for images, audio, etc. • Uses modes like "rb", "wb". Example: with open("file.bin", "wb") as f: f.write(b"Hello") <table><tr><th>Text File / Text Mode</th><th>Binary File / Binary Mode</th></tr><tr><td>Text file → Readable format</td><td>Binary file → Not readable format</td></tr><tr><td>Stores data as characters</td><td>Stores data as bytes</td></tr><tr><td>Text mode → "r", "w"</td><td>Binary mode → "rb", "wb"</td></tr></table>	Text File / Text Mode	Binary File / Binary Mode	Text file → Readable format	Binary file → Not readable format	Stores data as characters	Stores data as bytes	Text mode → "r", "w"	Binary mode → "rb", "wb"	6	L2	CO4
Text File / Text Mode	Binary File / Binary Mode										
Text file → Readable format	Binary file → Not readable format										
Stores data as characters	Stores data as bytes										
Text mode → "r", "w"	Binary mode → "rb", "wb"										

		Used for .txt, .csv files	Used for images, audio, video files			
Module – 4						
Q.7	a	Explain the use of random and time modules in Python. Develop a program that simulates a simple stopwatch that records random time intervals and calculates the average elapsed time.		8	L3	CO3
		- The random module is used to generate random numbers. - It is imported using import random. - randint(a, b) generates random integer between a and b. - It is used in games and simulations. - The time module is used to measure and control time. - It is imported using import time. - time() gives current time in seconds. - sleep(seconds) pauses the program for given seconds. Program: <pre>import random import time intervals = [] for i in range(5): start = time.time() delay = random.randint(1, 3) time.sleep(delay) end = time.time() intervals.append(end - start) average = sum(intervals) / len(intervals) print("Intervals:", intervals) print("Average time:", average)</pre>				
	b	Explain the concept of namespaces in Python. Develop program to illustrate how variable lookup follows the LEGB (Local, Enclosing, Global, Built-in) rule.		8	L3	CO3
		- Namespace is a container that stores variable names and their values. - It avoids name conflicts in program. - Each function and module has its own namespace. Python follows LEGB rule to search variables: - L – Local namespace (inside current function) - E – Enclosing namespace (inside outer function) - G – Global namespace (defined outside functions) - B – Built-in namespace (predefined names) - Python searches variables in this order: Local → Enclosing → Global → Built-in. Program: <pre>x = "Global"</pre>				

	<pre>def outer(): x = "Enclosing" def inner(): x = "Local" print(x) inner() outer() • Output is Local because Python checks local namespace first.</pre>				
c	Differentiate between class attribute and instance attribute with suitable program segments. • Class attribute is defined inside the class. • It is shared by all objects of the class. • Same value for every object. • Instance attribute is defined inside constructor using self. • It is different for each object. • Each object has its own copy. Program: <pre>class Student: college = "ABC College" def __init__(self, name): self.name = name s1 = Student("Ram") s2 = Student("Shyam") print(s1.college) print(s1.name) • college is class attribute. • name is instance attribute.</pre>	4	L2	CO5	
OR					
Q.8	a	Develop python script to create a module utilities.py with functions for square, cube, and factorial of a number. Import it in another file using all three import variants. Demonstrates the usage of each of the imported function. • A module is a file containing Python functions. • It helps in code reusability and organization. <pre>utilities.py def square(n): return n*n def cube(n):</pre>	8	L3	CO3

	<pre> return n*n*n def factorial(n): if n == 0: return 1 return n * factorial(n-1) Import using three variants: 1. Import whole module import utilities print(utilities.square(4)) 2. Import specific functions from utilities import square, cube print(square(3)) 3. Import all functions from utilities import * print(factorial(5)) </pre>			
b	Develop a custom module having function which calculates factorial of a number. Import this custom module to a program to calculate binomial coefficient. • Create a module that calculates factorial. • Import it to compute binomial coefficient. • Formula: $nCr = n! / (r! * (n-r)!)$ <pre> factorial_module.py def factorial(n): if n == 0: return 1 return n * factorial(n-1) Main program: import factorial_module n = 5 r = 2 result = factorial_module.factorial(n) / (factorial_module.factorial(r) * factorial_module.factorial(n-r)) print("Binomial Coefficient:", result) </pre>	8	L3	CO3
c	Explain the difference between ‘is’ and ‘==’ operators using immutable objects. Difference Between ‘is’ and ‘==’ (4 Marks) • == compares values of two objects. • It returns True if values are equal. • is compares memory location of two objects. • It returns True if both refer to same object.	4	L2	CO5

		Example: a = 10 b = 10 print(a == b) print(a is b) • == checks value equality. • is checks object identity.			
Module – 5					
Q.9	a	Create a Python class Point with attributes x and y. Demonstrate sameness using 'is' operator, and show the effect of mutability when modifying one reference. • A class is a blueprint to create objects. • Attributes store data inside an object. • The is operator checks whether two references point to same object. • Objects are mutable if their data can be changed. Program: <pre>class Point: def __init__(self, x, y): self.x = x self.y = y p1 = Point(2, 3) p2 = p1 print(p1 is p2) p2.x = 10 print(p1.x) print(p2.x)</pre> • p1 is p2 returns True because both refer to same object. • Changing p2.x also changes p1.x due to mutability. • Both references point to same memory location.	8	L3	CO5
	b	Explain the need for exception handling in Python. Develop a program to illustrate: try, except, else, and finally blocks. • Exception handling prevents program from crashing. • It handles runtime errors. • try block contains risky code. • except handles the error. • else runs if no error occurs. • finally always executes.	8	L3	CO5

		Program: <pre> try: a = int(input("Enter a number: ")) b = int(input("Enter another number: ")) result = a / b except ZeroDivisionError: print("Cannot divide by zero") except ValueError: print("Invalid input") else: print("Result is:", result) finally: print("Program finished") </pre> • If error occurs, except block runs. • If no error, else block runs. • Finally block runs in all cases.			
		c What is operator overloading? Illustrate with example using __add__(). • Operator overloading means giving special meaning to operators. • In Python, it is done using special methods. • __add__() is used to overload + operator. Program: <pre> class Number: def __init__(self, value): self.value = value def __add__(self, other): return Number(self.value + other.value) n1 = Number(5) n2 = Number(10) n3 = n1 + n2 print(n3.value) </pre> • Here + operator adds values of two objects. • It uses __add__() method.	4	L2	CO5
OR					
Q.10	a	Develop a program to illustrate polymorphism by defining a common interface method in two different classes. • Polymorphism means same method name behaves differently. • Different classes can have same method name.	8	L3	CO5

	Program: <pre> class Dog: def speak(self): print("Dog barks") class Cat: def speak(self): print("Cat meows") def make_sound(animal): animal.speak() d = Dog() c = Cat() make_sound(d) make_sound(c) </pre> - Both classes have speak() method. - Same interface but different behavior.			
b	Outline the difference between pure functions and modifiers. Develop a program code illustrating both using a class BankAccount. - Pure function does not modify original object. - It returns new value. - Modifier function changes object state. Program: <pre> class BankAccount: def __init__(self, balance): self.balance = balance def get_balance(self): return self.balance def deposit(self, amount): self.balance += amount acc = BankAccount(1000) print(acc.get_balance()) acc.deposit(500) print(acc.get_balance()) </pre> - get_balance() is pure function. - deposit() is modifier because it changes balance.	8	L3	CO5
c	Explain the role of finally clause with an example. Finally block always executes.	4	L2	CO5

- | | | | | |
|--|--|--|--|--|
| | <ul style="list-style-type: none">• It runs whether exception occurs or not.• Used to release resources like files. <p>Example:</p> <pre>try:
 f = open("file.txt", "r")
 print(f.read())
except FileNotFoundError:
 print("File not found")
finally:
 print("Execution completed")</pre> <ul style="list-style-type: none">• Finally block ensures important code runs at the end. | | | |
|--|--|--|--|--|

